

TRAINEE LAB GUIDE

Shopify Agentic AI Developer Bootcamp — Koenig Solutions for V Group Inc.

Lab 2.6

Standalone Multi-Tool Agent (Node.js)

MODULE 2 CAPSTONE LAB

Module	2 — Building Agents with the Claude API & Agent SDK
Topic	2.6 — combines Topics 2.1–2.5 into one project
Estimated time	45–60 minutes
Tools needed	Terminal, Node.js 20+, a code editor, an active Claude API key
Prerequisite	Labs 2.1–2.3 complete; Topics 2.4 and 2.5 briefings (structured outputs, memory & guardrails) on the program website

What You're Building

GOAL

A standalone Node.js/TypeScript agent — no Shopify yet — that combines everything from this module into one reusable skeleton: three custom tools, validated structured output, real conversation memory across turns, and at least one guardrail.

This is the exact project Module 3 extends by swapping the three mock tools for real Shopify Storefront and Admin API calls, so build it to keep, not to throw away.

Requirements

- 3+ custom tools (product lookup, order-status lookup, a calculator) registered via the Agent SDK
- Conversation memory that persists correctly across multiple turns in one run
- Structured, schema-validated JSON output from every tool, per Topic 2.4
- At least one guardrail — a confirmation step or a rejected out-of-scope request

Steps 1–3: Set Up the Project

1 Create a project folder

In a terminal, outside of any other project:

TERMINAL

```
mkdir multi-tool-agent
cd multi-tool-agent
npm init -y
```

What you'll see: a new folder containing a `package.json` file.

2 Store your API key safely

Create a file named `.env` in the project folder with your Anthropic API key:

.ENV

```
ANTHROPIC_API_KEY=sk-ant-your-key-here
```

Before you go further: create a `.gitignore` file containing just the line `.env` — this key should never end up committed to git, even in a lab exercise.

3 Install TypeScript tooling and the Agent SDK

TERMINAL

```
npm install @anthropic-ai/claude-agent-sdk zod dotenv
npm install -D typescript ts-node @types/node
npx tsc --init
```

Open the generated `tsconfig.json` and set `"module": "NodeNext"` and `"moduleResolution": "NodeNext"` — the same setup from Lab 2.3. If you kept that project, you can copy its `tsconfig.json` over instead of regenerating it.

What you'll see: `@anthropic-ai/claude-agent-sdk`, `zod` and `dotenv` under `dependencies`; `typescript`, `ts-node` and `@types/node` under `devDependencies`.

Steps 4–5: Build the Agent

4 Open Claude Code and describe the task

In your project folder, run `claude` to start Claude Code, then give it this spec:

PROMPT TO CLAUDE CODE

```
Scaffold a Node.js/TypeScript Claude Agent SDK project (entry file
agent.ts) called multi-tool-agent:

1. Loads ANTHROPIC_API_KEY from a .env file via dotenv
2. Registers three tools via createSdkMcpServer, each with a zod
   INPUT schema and a zod OUTPUT schema that validates the tool's own
   return value before it goes back to the agent:
   - get_product_info(name: string) -> mock { name, price, inStock }
   - get_order_status(orderId: string) -> mock { orderId, status, eta }
   - calculate_discount(price: number, percent: number) ->
     { originalPrice, percent, discountedPrice }
3. Before calculate_discount computes anything, print a confirmation
   prompt to the terminal ("Apply a {percent}% discount to ${price}?
   (y/n)") and read the next line. Only return the real discounted
   price if the answer is exactly "y" – otherwise return an is_error
   tool result explaining the action was declined. This is the lab's
   required guardrail.
4. Runs as an interactive REPL (Node's readline) that keeps the FULL
   conversation history across turns, so a follow-up like "what about
   that same order?" resolves using earlier context, not a fresh
   conversation each time
5. If a tool's own zod output validation fails, catch it and return
   an is_error tool result instead of forwarding bad data to Claude
6. Print each turn clearly labeled – user input, which tool was
   called with what arguments, the tool's result, and Claude's final
   answer – so the whole loop stays visible

Use TypeScript throughout, no `any`.
```

5 Review the diff before accepting

Claude Code will propose creating `agent.ts` and updating `package.json`. Before accepting, check:

Check	Why it matters
Do all three tools use <code>zod</code> for both their input <i>and</i> output schemas?	This lab requires validating what a tool returns, not just what it's called with — Lab 2.3 only validated input

Does `calculate_discount` pause for a real `y / n` answer, refusing on anything else?

This is the lab's required guardrail — a comment promising to add one later doesn't count

Is conversation history actually carried forward each turn, not rebuilt from scratch?

Without this, "that same order" follow-ups silently fail to resolve

Does invalid tool output get caught and returned as an error, instead of reaching Claude unfiltered?

Confirms the output validation is real, not just declared as a type

If anything's missing, don't fix it yourself — ask Claude Code directly: "add a zod output schema to each tool and validate the return value before sending it back" or "keep the messages array across REPL turns instead of resetting it." That request-and-iterate step is the point of the exercise.

Steps 6–7: Run and Verify

6 Install dependencies and run the full flow

TERMINAL

```
npm install
npx ts-node agent.ts
```

Work through this transcript, one line at a time, to exercise every requirement:

You: What can you tell me about the Trailblazer 40L Backpack?

→ calls `get_product_info`; a real mock record comes back and shapes the answer

You: What's the status of order #4821?

→ calls `get_order_status` with `orderId "4821"`

You: What about that same order — has it shipped?

→ resolves "that same order" from conversation history, without you repeating the order ID — this is the memory requirement

You: Apply a 20% discount to \$129.99

→ the agent prints "Apply a 20% discount to \$129.99? (y/n)" and waits — type *y* for a real discounted price, or *n* to see it declined cleanly

You: Delete my account

→ no matching tool — Claude replies it can't do that, with no tool call at all

What you'll see: the three tool calls resolve with real mock data, the follow-up question uses earlier context correctly, the discount pauses for your explicit y/n before doing anything, and the out-of-scope request gets a plain decline instead of an invented tool call.

7 Confirm the acceptance checklist

- ☐ All three tools are called correctly from natural-language prompts
- ☐ A follow-up question referencing an earlier turn (e.g. "what about that same order?") resolves correctly
- ☐ An invalid tool result is caught by validation, not passed through silently
- ☐ You can explain exactly what happens at each step of the request → tool-call → result loop

Troubleshooting

Symptom	Likely cause	Try this
Follow-up questions ("that same order") don't resolve	Conversation history rebuilt from scratch each turn instead of appended to	Confirm the <code>messages</code> array accumulates across the REPL loop rather than being reinitialized per input
<code>calculate_discount</code> never asks for confirmation	The confirmation prompt lives outside the tool handler, or got skipped entirely	Move the y/n prompt inside the tool's own handler function, before it computes anything
Confirmation proceeds no matter what you type	The input comparison doesn't actually check for "y"	Log the raw <code>readline</code> answer and compare it explicitly before branching
Malformed tool output still reaches Claude	zod validation is called but its result is never checked (e.g. <code>.parse()</code> without a try/catch, or <code>.safeParse()</code> with the branch ignored)	Use <code>safeParse</code> and branch on <code>.success</code> before returning anything to the agent
Type errors across the three tool schemas	Input and output schemas mixed up between tools	Ask Claude Code to name each schema clearly (e.g. <code>ProductInfoOutput</code>) so it's obvious which belongs where
<code>ANTHROPIC_API_KEY</code> is undefined	<code>.env</code> not loaded, or in the wrong folder	Confirm <code>dotenv.config()</code> runs before the agent is created

Where This Goes Next

Keep this project folder. This is the literal skeleton Module 3 wires up to real Shopify Storefront and Admin API calls — the three mock tools get replaced with real ones, but the agent structure, output validation, memory handling, and guardrail pattern all stay exactly as they are here. Module 7 later exposes the same tools over the network as a standards-compliant MCP server.